Group 2 - Statistics UG Practitioners Challenge 2026

Jasmine Charize Carpiz[1], Alyssa Siew Ching Yi[2], Bhakti Soni[3], Tiffany Vooi[4], and Yu See Loi[5]

[1]j.c.carpiz@lse.ac.uk
[2]a.c.siew@lse.ac.uk
[3]b.soni@lse.ac.uk
[4]t.x.vooi@lse.ac.uk
[5]y.s.loi@lse.ac.uk

March 4, 2026

1 Executive Summary

This research paper aims to discuss the performance results of the training of two neural networks, namely the Multi-Layer Perceptron (MLP) and the Long-Short Term Memory (LSTM). Both networks used simulated European call option prices for training.

2 Methodology

2.1 Simulating the Financial Data

We simulated 27 million rows of European call option prices, resulting from 9 strike prices, expiry dates ranging across 5 years, and 50,000 scenarios simulated across 12 months. The VIX (Volatility Index) is used to model volatility and fit to a t-distribution. The risk-free rate is the 6-month average of the 1-year Treasury rate, at 3.64 %. Stock prices were simulated using Geometric Brownian Motion. The Black-Scholes formula was then used to calculate the call prices. These data were then used to train the neural networks.

2.2 Standardisation

To ensure that the performance comparison of the neural networks is fair, the setup is as follows:

- Data split: 80/10/10 ratio split into training, validation, and test sets
- Max epochs in training: 30
- Max epochs in final model: 100
- Number of trials for optimisation: 50
- Patience: 15
- Use of early stopping
- Optimiser: Adam

2.3 Bayesian Optimisation via OPTUNA

A key component that was crucial to both models was selecting the most effective hyperparameters and how to optimise this decision-making process; inappropriate choices can lead to severe overfitting in deep neural networks. During our research, we came across two key methods:

- Grid Search
- Bayesian Optimisation via Optuna

Initially, the grid search method seemed like the optimal as it exhausts every possible combination of hyperparameters. However, this combined with our large dataset meant much longer run times, which we felt was inefficient. Hence, we looked at Bayesian optimisation, which as demonstrated by Brudere [1], provided lower run times. This allowed us to optimise the number of neurons, learning rate, and batch size more effectively than random or grid searches, resulting in more accurate results/ error metrics, with drastically reduced run times.[2]

Long Short-term Memory

Particularly in the case of Long Short-Term Memory (LSTM) Neural Networks, the increased computational complexity means traditional Grid Search was inefficient for our sample sizes; exhaustive grid search scales poorly with dimensionality. Consequently, using Bayesian Optimization helps navigate the parameter space, while drawing on methods for hyperparameter tuning and regularization strategies to mitigate overfitting.[3][4]

2.4 Multi-Layer Perceptron Neural Network

Many research papers have used MLPs as a way of predicting price and capturing the "complex, non-linear relationships" [5] that our simulated financial data demonstrates. This is due to the use of non-linear activation functions present in the hidden layers of the network. Our model uses Rectified Linear Unit (ReLU) activations throughout and it was trained under a supervised learning framework, where weights are iteratively updated by minimising the error between predicted and actual call prices across the training set.

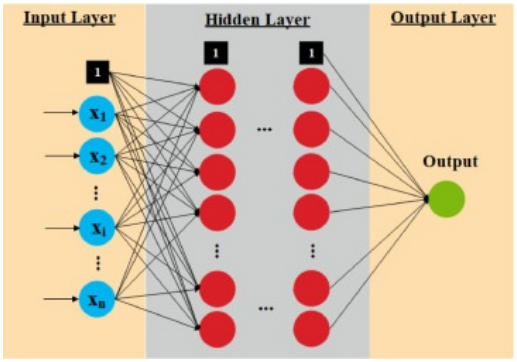


Figure 1: Structure of MLP
[5]

Prior to training, both the input features and the target variable were standardised using a RobustScaler, which centres each variable on its median and scales by the interquartile range. This approach is preferable to standard normalisation in the presence of outliers, which are common in financial data. Scaling was fitted exclusively on the training partition and applied to the remaining splits to prevent data leakage.

Given the scale of the full simulated dataset, a random 15% sample (around 4 million rows) was drawn to keep processing time manageable. This sample was partitioned into two non-overlapping pools: a hyperparameter search set (30%) and a retraining set (70%). Each pool was then independently divided into training, validation, and test subsets using the standard ratio mentioned earlier.

After the hyperparameters were tuned, the optimal configuration was used to retrain the model from scratch on the larger retraining set, ensuring that the final evaluation reflects performance on unseen data. The code for the MLP network can be found here [6].

2.5 Long-Short Term Memory Neural Network

While MLPs are effective at capturing non-linear relationships, financial data often contains sequential dependencies that standard feed-forward architectures may overlook. For this reason, we use a Long Short-Term Memory (LSTM) network, a specialized type of Recurrent Neural Network (RNN) designed to handle long-term dependencies through a unique gating mechanism.[7] Our model processes input features including:

- Asset price (S)
- Strike price (K)
- Time to maturity (T)
- volatility (σ)

to predict the corresponding call price.[8]

The core strength of the LSTM lies in its ability to regulate the flow of information using forget, input, and output gates, which helps the model decide which historical data points are relevant for the final price prediction. In our implementation, we utilized a MinMax Scaler to transform the input features and target variables into a bounded [0, 1] range. This normalization is critical for LSTM architectures to prevent gradient saturation, ensuring features with larger absolute magnitudes, such as the asset price (S), don't numerically overwhelm smaller values like volatility (σ).[9]

The model was trained using supervised learning, where the Mean Squared Error (MSE) loss function was minimized through iterative weight adjustments via the Adam optimizer. This lead to refined performance on validation data, which was consistent with high-precision financial modelling workflows.[4] The code for the LSTM can be found here [10].

3 Results

3.1 MLP

Both final models were evaluated relative to a targeted sample of the data using standard metrics for regression-based financial forecasting with neural networks [3], as seen below in the following tables.

The MLP network performed very well, achieving RMSE of less than 3% and MAE of less than 2%.

Metric	Bayesian Optimisation
Val loss	0.000004
Test R^2	0.999998
Test MAE	0.0150
Test RMSE	0.0283
Test MAPE (%)	0.1946
Optimal hyperparameters search time (s)	2584

Table 1: MLP - Key Model Performance Metrics

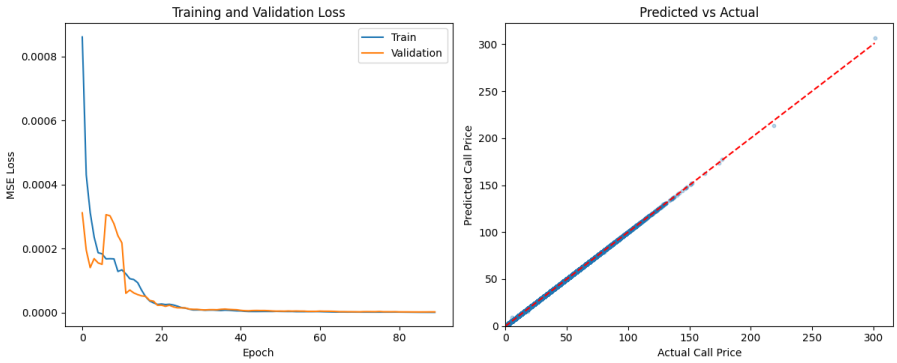


Figure 2: MLP Model Performance

The Bayesian optimisation strategy yielded strong model performance for the MLP, with a near-perfect R^2 score, which indicated that the model explains variance almost perfectly for the strike price. The exceptionally low validation loss, along with a low MAE and RMSE, confirm that the model's predictions deviate only minimally from the true values. Taken together, these results suggest that the MLP was highly effective in identifying a model configuration that generalises well to unseen data.

3.2 LSTM

The results indicate that LSTM serves as a high-precision proxy for the Black-Scholes pricing surface, achieving near-perfect correlation with theoretical values.

Metric	Bayesian Optimisation
Val loss	0.000149
Test R^2	0.987278
Test MAE	0.134590
Test RMSE	0.209137
Test MAPE (%)	0.014486
Optimal hyperparameters search time (s)	12228

Table 2: LSTM - Key Model Performance Metrics

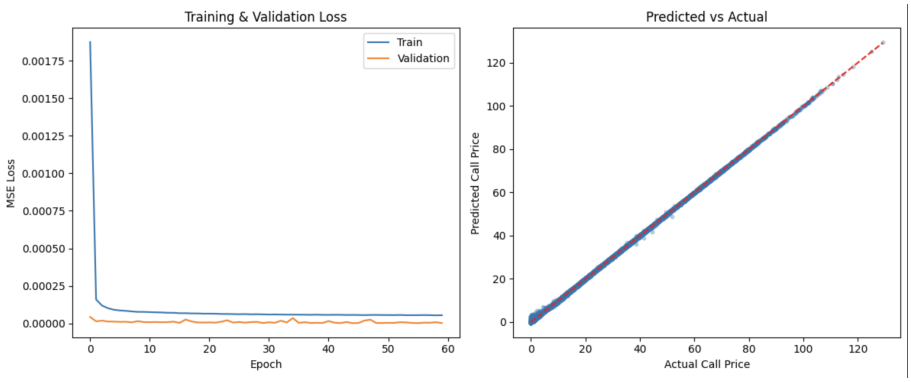


Figure 3: LSTM Model Performance

The exceptional R^2 score (0.987278) indicates that it is a highly reliable proxy for the underlying Black-Scholes pricing mechanism. The MAE (0.134590) and the RMSE (0.209137) being relatively close proves that the model is not suffering from large extreme errors/ outliers in its prediction. MAE shows that, on average, the model's prediction is off by only 0.13 units.

What stands out is the extremely low validation loss (0.000149), it suggests that the model achieved high numerical stability during the training process, with predicted prices deviating minimally from the target ones. Additionally, the low MAPE of (1.4466%) confirms that the error is small not only in absolute terms, but also relative to the magnitude of the prices being predicted.

4 Discussion

4.1 Evaluation

Both networks gave similar R^2 results, almost to 1 which means that the predicted values almost matches exactly with the actual values. However, comparing the test MAE and test RMSE, the MLP significantly outperformed the LSTM by around 90% as shown in the table below

Metric	MLP	LSTM
MAE	0.0150	0.1346
RMSE	0.0283	0.2091
R^2	0.9999	0.9873

Table 3: Result Comparison Table

This can be due to the fact that the call option prices depend on a deterministic mathematical formula from Black-Scholes, and the LSTM is not optimised for function approximation of static inputs. The simulated European call prices are not time-series data, but have an independent and static feature. Hence the complexity of the LSTM model is not suitable for this particular type of data, whereas the MLP is built exactly for this kind of input-output mapping, where the call prices are determined by the various inputs and is memoryless.

4.2 Limitations

Due to time and resource constraints, the hyperparameter tuning stage for each of the neural networks did not include optimising for other common and relevant hyperparameters. Examples of these would include: patience; the number of maximum epochs; activation function; the use of early-stopping; the choice of optimiser; depth (number of layers); and width (number of neurons in a single layer). Vanilla Bayesian optimisation was used as a result of such constraints, but other methods, in particular BOHB due to its versatility and robustness, are proposed to be more suitable as a default. [11] Moreover, this meant there were also other limitations in the optimisation process, such as in the sample size used for training.

When attempting to use neural networks for non-personal use, such as in this case for stock price predictions, the process of hyperparameter tuning and optimisation becomes paramount, as it has been shown that un-tuned models consistently perform poorly across a range of dataset types when compared to their tuned counterparts[12].

Although the MLP model has been able to perform very well, there are reasons it may not perform that well in a professional context. The results are based on only 15% of the full dataset, so performance on the full data is unknown. It is also important to consider that the data was simulated rather than historic so actual performance could differ due to market microstructures, jumps, or volatility clustering which are not captured in simulation. Another key point is that a single train/test split means there is no cross-validation to confirm stability of results. We've run this model numerous times and have obtained slightly varying results (+/- 0.02 for MAE and RMSE) so more advanced methods could be used for further consolidate it's performance.

4.3 Bonus Question: Alternative Neural Networks

A suitable alternative neural network is the Gated Recurrent Unit (GRU). Its recurrent neural network (RNN) structure is similar to that of the LSTM, but simplified to consist only of two main gates, the update and reset gates (rather than the input, output and memory gates in LSTMs). It is like LSTMs in that GRUs can address the vanishing gradient issue of traditional RNNs. Additionally, studies have shown [13] that GRUs perform comparatively to LSTMs - particularly in learning time-series dependencies - making it an appropriate alternative for predicting call option prices. However, the lack of a separate memory cell state (included in LSTMs) means that GRUs tend to have a reduced run time but are also less capable of handling larger data sets. Furthermore, this 'missing' memory gate result in the model's content being exposed fully rather than in a controlled manner like LSTMs. The subtle but important differences between the two models means that whilst they are closely related, it is difficult to conclude which would perform better in general.

Unfortunately, since GRUs and MLPs have vast structural differences, accurate comparisons cannot be made without empirical results. In theory, GRUs are far better suited to capture time-series dependencies but MLPs are simpler, easier and faster to train.

Other possible neural networks include the convolutional neural network (CNN) and the graph neural network (GNN). However, the CNN is typically used for image generation and the GNN is more often used on graphical structures, which make them less suitable for call option price predictions.

Radial Basis Function Network (RBF) can also be used as it is similar to the MLP but uses

radial basis functions as activation functions instead of ReLU/sigmoid. Hence it works well for function approximation tasks like option pricing. [14]

5 Conclusion

Although the LSTM model is not suitable for the memoryless dataset used in this research, a performance analysis on real financial markets data should be used to train it as it is most likely to perform better on a time-series dataset as demonstrated by research papers where the model was trained on the TSLA data set[8]. Otherwise in this context of analysing various inputs to predict an output, the Multi-Layer Perceptron performs significantly better than the Long-Short Term Memory neural network.

Bibliography

[1] K. D. Brūdere, "Bayesian hyperparameter optimisation for neural networks," 2025, last accessed 25 February 2025. [Online]. Available: https://www.theseus.fi/handle/10024/890082

[2] Geeksforgeeks, "Hyperparameter tuning," 2025. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/hyperparameter-tuning/?ref=hackernoon.com

[3] G. Tandberg, "Financial forecasting using lstm networks," 2015. [Online]. Available: https://wiki.ubc.ca/Course:CPSC522/Financial_Forecasting_using_LSTM_Networks

[4] K. Azar, "Stock price prediction lstm hyperparameter tuning," 2021. [Online]. Available: https://www.kaggle.com/code/kamyarazar/stock-price-prediction-lstm-hyperparameter-tuning

[5] S. Abolmakarem, F. Abdi, K. Khalili-Damghani, and H. Didehkhani, "A multi-stage machine learning approach for stock price prediction: Engineered and derivative indices," 2024, last accessed 25 February 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2667305324001236

[6] A. members of the group, "Python code for the mlp neural network with hyperparamter optimisation," 2026. [Online]. Available: https://colab.research.google.com/drive/1Gd8wu44kfpLRCDaPdKxi1SHMsHZeDBM2?usp=sharing

[7] J. Starmer, "Neural networks / deep learning," 2023. [Online]. Available: https://www.youtube.com/playlist?list=PLblh5JKOoLUIxGDQs4LFFD--41Vzf-ME1

[8] D. K. Vikas Deswal, "Hyperparameter tuning of the lstm model for stock price prediction," *International Journal of INTELLIGENT SYSTEMS AND APPLICATIONS IN ENGINEERING*, 2024.

[9] D. A. K. Rajneesh Chaudhary, "Advanced stock market prediction using long short-term memory networks: A comprehensive deep learning framework," 2025. [Online]. Available: https://arxiv.org/html/2505.05325v1#bib.bib1

[10] A. members of the group. [Online]. Available: https://colab.research.google.com/drive/1Pzo7Wukdr9wcfrOCr9FAokuE0ZMnk7n8?usp=sharing

[11] J. V. E. Frank Hutter, Lars Kotthoff, "Automated machine learning: Methods, systems, challenges," 2019.

[12] H. A. Kazi Ekramul Hoque, "Impact of hyperparameter tuning on machine learning models in stock price forecasting," 2021. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9643011

[13] Y. Bengio, K. Cho, J. Chung, and C. Gulcehre, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014. [Online]. Available: https://arxiv.org/abs/1412.3555

[14] Y. Wu, H. Wang, B. Zhang, and K. Du, "Using radial basis function networks for function approximation and classification," pp. 1–34, 2012. [Online]. Available: https://www.researchgate.net/publication/258403761_Using_Radial_Basis_Function_Networks_for_Function_Approximation_and_Classification